COLLEGE PROJECT DOCUMENTATION

LEGAL EASE

AI-Powered Legal Document Generator

PROJECT NAME	LegalEase
PROJECT TYPE	Artificial Intelligence / Web Application
FRONTEND	Streamlit
BACKEND	FastAPI
AI MODEL	Google Gemini 1.5 Pro

SUBMITTED BY

S.No.	Student Name	NM ID
1	Gayathri G	7AB55F06CCC551FBD514C89291AC423C
2	Dhanya Shri B	14CD7115AC734F7559ACA4341BB785EFA9b
3	Dhanalakshmi M	AD0C3ECF6D26047A7BC4D25FEDCFF614
4	Nikitha M	9B162A64D4DE0BABAE94A4E6867C728B

Submitted as part of the academic project work

TABLE OF CONTENTS

1. Project Overview

2. Project Description

3. Objectives

4. Prerequisites and Technologies

5. Project Workflow

6. System Architecture

7. Core Functionalities

8. Backend Development – FastAPI

9. AI Integration – Gemini

10. Frontend Development – Streamlit

11. Document Generation and Export

12. Deployment

13. Testing and Verification

14. Exploring LegalEase

15. Conclusion

1. Project Overview

LegalEase is an AI-powered legal document generation application designed to simplify the creation of structured legal documents. The application accepts user-specific details such as document type, parties, terms and effective dates, and generates editable legal content. The project combines a Streamlit frontend, a FastAPI backend and Google Gemini integration.

Key Features

- Generation of legal documents based on user inputs.
- Editable preview of generated content.
- Export of documents in TXT, DOCX and PDF formats.
- Structured formatting with headings, tables, logos and footers.
- Secure handling of the Gemini API key through environment configuration.

2. Project Description

LegalEase leverages generative AI to provide customizable and editable templates for a range of legal use cases, including employment contracts, lease agreements, NDAs and other agreements. The system is intended to make legal-document creation more accessible while maintaining a professional document structure and supporting multiple export formats.

Example Scenarios

- A startup founder generates an employment contract by entering role, compensation, confidentiality and related details.
- A freelancer generates an NDA by providing the parties, confidentiality scope and effective date.
- A landlord creates a residential lease agreement using property, tenant and lease-term information.

3. Objectives

Primary Objectives

- To simplify the creation of structured legal documents.
- To use generative AI for producing content from user-provided information.
- To provide an easy-to-use interface for entering legal document details.
- To allow users to preview and edit generated documents.
- To support document export in TXT, DOCX and PDF formats.
- To maintain professional formatting and branding in generated documents.

4. Prerequisites and Technologies

Prerequisites

- FastAPI framework knowledge.
- Google Gemini / Generative AI API familiarity.
- Streamlit frontend knowledge.
- Python programming proficiency.

- A configured development environment with required packages.

Technology Stack

- Python
- FastAPI and Uvicorn
- Streamlit
- Google Generative AI SDK
- Google Gemini 1.5 Pro
- Pydantic
- python-docx
- FPDF
- Pillow
- Requests
- python-dotenv

5. Project Workflow

The project is organized into a sequence of development activities covering model selection, architecture, core functionality, API integration, frontend development, deployment and testing.

Activity	Stage	Description
Activity 1	Model Selection and System Architecture	Select the generative AI model and define frontend, backend and AI components.
Activity 2	Core Functionalities Development	Implement document generation, formatting, editing and export features.
Activity 3	API Logic Integration	Configure FastAPI and connect the document-generation route to the Gemini generator.
Activity 4	Frontend Development	Build the Streamlit interface, dynamic preview and download options.
Activity 5	Deployment	Configure dependencies, environment variables and local deployment.
Activity 6	Finalization and Testing	Verify end-to-end functionality using different input combinations.

6. System Architecture

Architecture Components

- Frontend: Streamlit for user interaction, input, preview, editing and downloads.
- Backend: FastAPI for processing API requests and routing document-generation requests.
- AI Core: Gemini integration for generating legal document content.
- Formatting Modules: DOCX, PDF and TXT generation utilities.

The frontend collects document_type, parties, terms and dates. The backend receives the data through the /generate endpoint, validates the request and passes the information to the Gemini document generator. The generated content is returned to the frontend for preview, editing and export.

7. Core Functionalities

Legal Document Generation

The system generates legal documents such as contracts, agreements, NDAs, lease agreements and employment-related documents based on user inputs.

Document Output Formatting

Generated text is converted into structured TXT, DOCX and PDF outputs. DOCX formatting can include a logo, headings, a terms table and a footer. PDF formatting supports headers, footers, headings and bullet-style terms.

User Interaction

The Streamlit interface accepts user preferences and reflects the entered information in the generated legal content.

Editable Preview and Download

Users can preview generated content, edit it when required and download the final document in TXT, DOCX or PDF format.

8. Backend Development – FastAPI

FastAPI Application

The backend initializes a FastAPI application, provides a root endpoint for checking that the service is running and includes modular routes.

Document Request Model

A Pydantic DocumentRequest model is used to validate incoming document-generation data.

Generate Endpoint

The POST /generate endpoint accepts document information and calls the GeminiDocumentGenerator to return generated legal content.

Backend Responsibilities

- Receive structured input from the frontend.
- Validate document-generation information.
- Route requests to the Gemini document generator.
- Return structured generated content to the frontend.

9. AI Integration – Gemini

Model Selection

The project documentation specifies Gemini 1.5 Pro as the selected generative AI model for legal document generation.

Integration Process

- A structured prompt is created from the user's document type, parties, terms and dates.
- The prompt is sent to Gemini through the Generative AI integration.
- The generated text is returned to the FastAPI backend.
- The frontend displays the generated content for preview and editing.

Environment Configuration

The Gemini API key is stored in a .env file so that the application can access the service without placing the key directly in the source code.

10. Frontend Development – Streamlit

User Interface

- Input fields for document type and effective date.
- Text areas for parties and terms.
- Generate Document button.
- Editable document preview.
- Download options for TXT, DOCX and PDF.

Dynamic Preview

After generation, the content is displayed dynamically in a styled preview area. Users can select the editing option and modify the generated text before saving it.

11. Document Generation and Export

TXT Export

The application can save generated legal content as plain text.

DOCX Export

The DOCX generator uses python-docx to create a formatted Word document. The documented formatting includes a logo, titles, paragraph formatting, a terms table and footer.

PDF Export

The PDF generator uses FPDF to produce a formatted document with a centered logo, headings, bullet-style terms and header/footer elements.

HTML Preview

The generated content can also be converted into styled HTML blocks for inline display in the Streamlit interface.

12. Deployment

Local Deployment

The project documentation describes local deployment using FastAPI as the backend and Streamlit as the frontend.

Deployment Steps

- Create and activate a virtual environment.
- Install dependencies using requirements.txt.
- Configure the Gemini API key in the .env file.
- Launch the FastAPI backend using Uvicorn.
- Launch the Streamlit frontend.
- Test document generation, editing, preview and downloads.

Future Deployment Options

The source documentation identifies platforms such as Render, Railway, Deta, Streamlit Community Cloud, Fly.io or a VPS as possible deployment options.

13. Testing and Verification

Functional Testing

- Enter document type, parties, terms and effective date.
- Generate the legal document using the Gemini integration.
- Verify the generated preview.
- Edit the generated content.
- Download the document in TXT, DOCX and PDF formats.
- Check logo and footer placement.
- Check bullet-point and terms formatting.
- Verify that the entered terms are included in the generated document.

14. Exploring LegalEase

Document Type

Users select the type of legal document they want to generate. Examples include Agreement, Contract, NDA, Lease Agreement and Employment Offer Letter.

Parties Involved

Users provide the names and roles of the individuals or organizations involved in the document.

Terms and Conditions

Specific clauses can be entered using semicolons to separate individual terms or bullet points.

Effective Date

The effective date specifies when the agreement becomes applicable.

Editing

After a document is generated, the user can select the edit option and modify the generated content.

Saving

The completed document can be downloaded and saved in TXT, DOCX or PDF format.

15. Conclusion

LegalEase demonstrates how artificial intelligence can be integrated with a user-focused web application to simplify legal document creation. The project combines natural language generation, structured inputs, document formatting and multiple export options in a single workflow.

The project provides users with a way to generate, preview, edit and save legal documents without requiring them to manually build the document structure from the beginning. Its architecture separates the Streamlit frontend, FastAPI backend, Gemini AI integration and document-formatting utilities, providing a clear basis for further development.

Future enhancements identified in the project material include deeper contract analysis, integration with legal databases and personalized recommendations. The documented system therefore provides a foundation for further development of AI-assisted legal information and document workflows.